تنمية المراجعة والتلخيص الأكاديمي

ASE - Chapter 5 - Software Design

Source: Approach & Single, Software Design (2nd ed.) — slide deck CH05_Software_Design.pdf (99 slides) + printed book pp. 202–251. Official Exercises 5.1–5.24 are on **slides 87–99**.

Format: EL = model answer in English; one short Arabic justification line after each answer. Every answer cites its slide/book source. Blank lines are inserted between blocks (85% rule).

Reference diagrams (review before solving)



The six steps of object-oriented design (Fig. 5.1)



PART A — [5.24-Exercises 5.1] (Approx. 5 Questions)

Every official exercise question from slides 87–99, each with a full exam-grade model answer.

Exercise 5.1 — What is design? Describe the difference between conceptual design and technical design.
Model answer: Design is the process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient detail to permit its physical realization. It software it is the activity that follows requirements and produces a blueprint for how the system will be built. **Conceptual design** describes what the system does at the level of its users — its components, their responsibilities, and how they collaborate — without any commitment to code, which classes are offered, the timing of events, and how inputs and screens look. **Technical design** describes how the system will be realized: hardware configurations, software specifications, network architecture, system I/O, software architecture and network architecture — in short, anything that translates the requirements into a solution to the customer's problem. Conceptual design is thus the customer facing, requirements oriented view of the system, while technical design is the developer facing, solution oriented view.

Authority: Conceptual design is the process of applying various techniques and principles for the purpose of defining a device, a process or a system in sufficient detail to permit its physical realization. It software it is the activity that follows requirements and produces a blueprint for how the system will be built. **Conceptual design** describes what the system does at the level of its users — its components, their responsibilities, and how they collaborate — without any commitment to code, which classes are offered, the timing of events, and how inputs and screens look. **Technical design** describes how the system will be realized: hardware configurations, software specifications, network architecture, system I/O, software architecture and network architecture — in short, anything that translates the requirements into a solution to the customer's problem. Conceptual design is thus the customer facing, requirements oriented view of the system, while technical design is the developer facing, solution oriented view.

Exercise 5.2 — Discuss the objectives of software design. How do we transform an informal design to a detailed design?
Model answer: The four design objectives are that the design must be **correct & complete, understandable, and the right level of abstraction**. The transformation from informal to detailed design is done through **refinement** and **enhancement** (Fig. 5). An **informal design outline** → **a refined design** → **a more formal design** → the

تصميم رسمي. التحكم التصميم يطابق أدوات إزالة وهذه تتم بتفاصيل **Control Coupling** ربطاً تصميم رسمي. التحكم التصميم يطابق أدوات إزالة وهذه تتم بتفاصيل **Control Coupling** ربطاً

Exercise 5.3 — What problems are likely to arise if module functions are duplicated?
Model answer: The real design objective is that the final product be **correct & complete, understandable, at the right level of abstraction**, and **maintainable**. The design process is thus a process of refinement and enhancement (Fig. 5). An **informal design outline** → **a refined design** → **a more formal design** → the

Exercise 5.4 — What is modularity? List the important properties of a modular system.
Model answer: Modularity is the property of a system of software that allows it to be **effectively managed**. A modular system consists of well-defined, manageable units with well-defined interfaces among them. **Strong coupling** is a problem with a system composed of modules that are highly dependent on one another. Two packages, say, to work in assignment 3, the programmer must understand the purpose of the other package well enough to know what services it provides to the rest of the system. **Weak coupling** is the opposite. Each module can be understood separately. **Cohesion** is a measure of how well a module fits together. A module with **low cohesion** tends to group together unrelated ideas and operations that have little to do with one another. **High cohesion** tends to group together ideas that are related to one another and that work together to provide well-bounded behavior. Modules with high cohesion tend to be more reusable than modules with low cohesion.

Exercise 5.5 — Define module coupling and explain different types of coupling.
Model answer: Coupling is the **measure of the degree of interdependence between modules** (from slide 17). **Low coupling** → modules are independent. **High coupling** → modules are highly dependent. Ranked — best → worst, and: **Data coupling** — modules communicate only by passing data; apart from that they are independent. **Stamp coupling** — a composite data structure is passed between modules. **Control coupling** — modules communicate by passing control information, usually flags set by one module and tested upon by another. **Common coupling** — modules share access to the same global data. **Content coupling** — one module directly references the contents of another module — the worst form, where one module changes a private item on a shared data area, or one module branches into the middle of another. Slide 17 also lists **Common coupling** as the worst case because the data of another or control flow is accessed without any restriction. Slide 18 adds **Pathological coupling** as the worst case, where a module depends on the internal implementation details of another module.

Exercise 5.6 — Explain why it is important to pay attention to data flow design.
Model answer: Data flow design is important because it helps designers understand the transformation of input data to output data. Data flow diagrams are the main representation used during structured analysis, and they can be refined into structure charts during structured design. Data flow diagrams can be used to represent a system or software at any level of abstraction. In fact, data flow diagrams may be partitioned into levels that represent increasing information flow and functional detail. Therefore, these levels of data flow diagrams provide a framework for implementing both top-down and bottom-up approaches to software design. The top-down approach starts at the highest level of the data flow diagram hierarchy and works its way down to the lowest level. A bottom-up approach starts at the lowest level and works its way up to the highest level. A complete software design will most likely contain a balanced combination of both approaches.

Exercise 5.7 — What is the difference between a flow chart and a structure chart?
Model answer: A flow chart describes the logic and sequence of control inside a single program — decisions, loops, and branches. A structure chart, by contrast, shows the relationship between modules in a system — the hierarchical module decomposition of the whole system — which module calls which, and what data and control information passes between them. In short: a flow chart describes how a single module works; a structure chart describes how modules work together.

Exercise 5.8 — Explain why it is important to pay attention to interface design.
Model answer: Interface design is important because a poorly designed interface can cause even the best-designed software to perform poorly. Poor interface design can cause errors, frustration, and even rejection of software that otherwise performs its functions correctly. Interface design is important because it helps the user utilize knowledge of the internal design. In short, a flow chart is an **intra-module** view (the "how" inside one module), while a structure chart is an **inter-module** view (the "how" between modules). Both are used during detailed design: the flow chart to design each module, and the structure chart to design how those modules fit together.

Exercise 5.9 — What problems are likely to arise if a module has low cohesion?
Model answer: A module with low cohesion (logical, temporal or coincidental) performs several unrelated functions. Such a module is hard to understand, because its purpose is unclear. It is hard to reuse, because it is unlikely that a module that does several unrelated things will perfectly match the requirements of another system. It is hard to modify, because a change to one function may affect the other functions in unpredictable ways. Low cohesion is also called **weak cohesion**.

Exercise 5.10 — Discuss the objectives of software design. What are the effects of module coupling and cohesion?
Model answer: The objective of modular design is to organize the program's modules so that they are easy to develop and later, easy to change — to deal with the size and complexity of programs by producing modules that are independently understandable and modifiable. Coupling and cohesion are two important criteria for modules for development and change, and why pre-existing code must still be understood, organized and parsed together. **Coupling** is a measure of interconnection among modules — a measure of the degree to which modules depend on each other. **Cohesion** is a measure of the degree to which the elements of a module belong together. **Low coupling** and **high cohesion** are the two primary objectives of a good design, because they support the two essential properties of modular design: **high-cohesion** modules focused on one task, making it easier to name, understand, test and reuse. The combination of **high cohesion** and **low coupling** is the ideal, because it supports the two most important properties a "tool" of seemingly trivial enhancements" from taking the project into. [5]

Exercise 5.11 — Explain the difference between coupling and cohesion. Which is more desirable?
Model answer: **Coupling** is the **measure of the degree of interdependence between modules**. **Cohesion** is the measure of the degree to which the elements within a module are functionally related. Their taxonomies (Fig. 7 and Fig. 8) are: **Coupling** (best → worst): **data** → **stamp** → **control** → **common** → **content**. **Cohesion** (best → worst): **functional** | **sequential** | **communicational** | **procedural** | **temporal** | **logical** | **coincidental**. Furthermore, good design is one that has **low coupling** and **high cohesion**, which means one well-defined job, hence is understandable, testable and reusable in isolation and **minimizes coupling** (each

Exercise 5.12 — If some existing modules are to be re-used in building a new system, which design strategy is used? Explain.
Model answer: When **top-down** strategy is used, because it begins from available/pre-written modules and then integrates them into the entire system. — which is exactly how a modifiable library of reusable modules is exploited. Existing modules are collected in a library (Fig. 13), so the design can be built by composing them from those available modules. **Bottom-up** strategy is used when the system is to be built from existing modules without referencing to a higher structure. Bottom-up testing is similar in practice, therefore, move means a bottom-up approach is used to design a bottom-up model of design. The choice between top-down and bottom-up depends on the situation.

Exercise 5.13 — Define module coupling and explain different types of coupling.
Model answer: Coupling is the **measure of the degree of interdependence between modules** (from slide 17). **Low coupling** → modules are independent. **High coupling** → modules are highly dependent. Ranked — best → worst, and: **Data coupling** — modules communicate only by passing data; apart from that they are independent. **Stamp coupling** — a composite data structure is passed between modules. **Control coupling** — modules communicate by passing control information, usually flags set by one module and tested upon by another. **Common coupling** — modules share access to the same global data. **Content coupling** — one module directly references the contents of another module — the worst form, where one module changes a private item on a shared data area, or one module branches into the middle of another. Slide 17 also lists **Common coupling** as the worst case because the data of another or control flow is accessed without any restriction. Slide 18 adds **Pathological coupling** as the worst case, where a module depends on the internal implementation details of another module.

Exercise 5.14 — Explain why it is important to pay attention to interface design.
Model answer: Interface design is important because a poorly designed interface can cause even the best-designed software to perform poorly. Poor interface design can cause errors, frustration, and even rejection of software that otherwise performs its functions correctly. Interface design is important because it helps the user utilize knowledge of the internal design. In short, a flow chart is an **intra-module** view (the "how" inside one module), while a structure chart is an **inter-module** view (the "how" between modules). Both are used during detailed design: the flow chart to design each module, and the structure chart to design how those modules fit together.

Exercise 5.15 — What is the difference between a flow chart and a structure chart?
Model answer: A flow chart describes the logic and sequence of control inside a single program — decisions, loops, and branches. A structure chart, by contrast, shows the relationship between modules in a system — the hierarchical module decomposition of the whole system — which module calls which, and what data and control information passes between them. In short: a flow chart describes how a single module works; a structure chart describes how modules work together.

Exercise 5.16 — Explain why it is important to pay attention to data flow design.
Model answer: Data flow design is important because it helps designers understand the transformation of input data to output data. Data flow diagrams are the main representation used during structured analysis, and they can be refined into structure charts during structured design. Data flow diagrams can be used to represent a system or software at any level of abstraction. In fact, data flow diagrams may be partitioned into levels that represent increasing information flow and functional detail. Therefore, these levels of data flow diagrams provide a framework for implementing both top-down and bottom-up approaches to software design. The top-down approach starts at the highest level of the data flow diagram hierarchy and works its way down to the lowest level. A bottom-up approach starts at the lowest level and works its way up to the highest level. A complete software design will most likely contain a balanced combination of both approaches.

Exercise 5.17 — What is the relationship between abstract data types and classes?
Model answer: An **abstract data type (ADT)** is the specification idea: a data type defined by the operations that can be performed on it, independent of any implementation. A **class** is the implementation mechanism in OOP that ideas: it binds data (attributes) and operations (methods) into one unit, hides its representation (encapsulation/information hiding), and serves as a blueprint from which objects can be instantiated. So a class is effectively a concrete, instantiable ADT: the ADT says what operations exist and what they mean, while the class says how they are stored and computed and in the object-oriented paradigm. Encapsulation is the enforcing mechanism — then the separation of the external aspects of an object from its internal implementation details.

Exercise 5.18 — Explain the following terms: (a) Object, (b) Class, (c) Method, (d) Message.
(a) Object — an entity able to save a state (information) and offering a number of operations (behaviors) to examine or affect that state; an object is characterized by its operations and a state that remembers their effect. **(b) Class** — a template for objects, describing the operations and the identity of the target object, the name of the requested operation and any other information needed; messages are often implemented as procedure or function calls. **Abstraction** — the elimination of the irrelevant and the amplification of the essentials, used to decide on what is important. **Inheritance** — the mechanism by which one class (subclass/derived classes) inherit state and behavior from a high level class (superclass/base class), created by abstracting common features from a set of classes. **Encapsulation** is the grouping together of data and operation and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients. **(c) Method** — the implementation of an operation. **(d) Message** — a request to an object to execute one of its methods.

Exercise 5.19 — Explain the design guidelines that can be used to produce "good quality" classes or reusable classes.
Model answer: Guidelines for good/reusable classes: (1) give each class a **single, coherent responsibility** — a class should represent one well-defined concept or thing (Square, Triangle), not a grab-bag. (2) **encapsulate** — keep attributes hidden and expose only a small, complete, well-named interface (information hiding). (3) **maximize cohesion** — every method should relate to the class's purpose; if a method feels out of place, it belongs elsewhere. (4) **minimize coupling** — reduce dependencies between classes; depend on abstractions (interfaces) rather than concrete classes. (5) **use inheritance only for genuine "is-a" generalization** and avoid deep, fragile hierarchies (the Shape/Square/Triangle example is the right case — abstract the common behavior). (6) **design for interface stable** and documented, once reuses depend on it, it's hard to change. **independent of implementation** — the interface should not leak implementation details. **Explain the type of each and how they are useful in solving a good design, with an example.**

Exercise 5.20 — Explain the design guidelines that can be used to produce "good quality" classes or reusable classes.
Model answer: Guidelines for good/reusable classes: (1) give each class a **single, coherent responsibility** — a class should represent one well-defined concept or thing (Square, Triangle), not a grab-bag. (2) **encapsulate** — keep attributes hidden and expose only a small, complete, well-named interface (information hiding). (3) **maximize cohesion** — every method should relate to the class's purpose; if a method feels out of place, it belongs elsewhere. (4) **minimize coupling** — reduce dependencies between classes; depend on abstractions (interfaces) rather than concrete classes. (5) **use inheritance only for genuine "is-a" generalization** and avoid deep, fragile hierarchies (the Shape/Square/Triangle example is the right case — abstract the common behavior). (6) **design for interface stable** and documented, once reuses depend on it, it's hard to change. **independent of implementation** — the interface should not leak implementation details. **Explain the type of each and how they are useful in solving a good design, with an example.**

Exercise 5.21 — List the points of a simplified design process.

Model answer: The simplified design process (Fig. 2) — book-fig 5.2(1–6) steps is: (1) **Create the use case model** — identify the actors interacting with the system, write the use cases and draw the use case diagram (5). (2) **Create the analysis model** — model the business aspects of the system by the flow of control from activity to activity. (3) **Draw the interaction diagram** — identify the objects for every use case and draw interaction diagrams for each use case. (4) **Draw the class diagram** — the model hierarchy and interfaces (association, dependency, aggregation, generalization). (5) **Draw the state chart / object diagram** (if required) — draw state charts for objects with significant dynamic behavior and object diagram (& composite) to show **component and deployment diagrams** — the static implementation view and the mapping of physical components to hardware — after which the **design documentation** is complete. [5]

Exercise 5.22 — Discuss the differences between object oriented and function oriented design.
Model answer: function oriented design is an approach in which the design is decomposed into a set of interacting units where each unit has a clearly defined function — the system is designed from a functional viewpoint, documented with DFDs, structure charts and pseudocode, and typically refined top-down as a tree of modules. **Object oriented design** is the design of software where the units are objects — encapsulated data items, instead the units that are to be manipulated by the program — it is **orthogonal** to function oriented design. Object oriented design is the design of software where the units are objects — encapsulated data items, instead the units that are to be manipulated by the program — it is orthogonal to function oriented design. Object oriented design is the design of software where the units are objects — encapsulated data items, instead the units that are to be manipulated by the program — it is orthogonal to function oriented design.

Exercise 5.23 — What documents should be produced on completion of the design phase?
Model answer: The document produced is the **Software Design Description (SDD)** per **IEEE STD 1016-1998** (slide 10). It is the blueprint for implementation. The SDD contains, in the recommended order: **Introduction** — purpose, scope, definitions, and overview of the SDD. **System overview** — how the software is structured to satisfy the requirements in the SRS — it is the blueprint for implementation. The SDD contains, in the recommended order: **Introduction** — purpose, scope, definitions, and overview of the SDD. **System overview** — how the software is structured to satisfy the requirements in the SRS — it is the blueprint for implementation. The SDD contains, in the recommended order: **Introduction** — purpose, scope, definitions, and overview of the SDD. **Design Description Information Content** — the **Design views** (slide 11): **Decomposition description**, the **Dependency description**, the **Interface description** and the **Detail description**. **Design Description Information Content** — the **Design views** (slide 11): **Decomposition description**, the **Dependency description**, the **Interface description** and the **Detail description**.

Exercise 5.24 — Can a system ever be completely "decoupled"? That is, can the degree of coupling be reduced or zero? If not, is there a need to employ software modules?
Model answer: No. For a system to work, its modules must collaborate, and collaboration requires some exchange — at a minimum the passing of data through a call or a shared interface — so some coupling always exists. Complete decoupling would mean the modules share nothing and do not interact, which describes not a real system. What we **minimize** is **coupling** — we reduce the amount and strength of it — but we do not eliminate coupling but to reduce it to **loose bonn — data coupling** — where modules communicate only by passing parameters, and **cohesion** — we aim for **high cohesion**, where each module has a single reason for coupling. Coupling can be reduced by controlling the number of parameters passed, avoiding passing unrelated data, keeping interfaces small and well-defined, and communicating through explicit interfaces rather than shared state.

PART B — [من قائمة المراجعة] (Approx. 5 Questions)

Exercise 5.24 — Can a system ever be completely "decoupled"? That is, can the degree of coupling be reduced or zero? If not, is there a need to employ software modules?
Model answer: No. For a system to work, its modules must collaborate, and collaboration requires some exchange — at a minimum the passing of data through a call or a shared interface — so some coupling always exists. Complete decoupling would mean the modules share nothing and do not interact, which describes not a real system. What we **minimize** is **coupling** — we reduce the amount and strength of it — but we do not eliminate coupling but to reduce it to **loose bonn — data coupling** — where modules communicate only by passing parameters, and **cohesion** — we aim for **high cohesion**, where each module has a single reason for coupling. Coupling can be reduced by controlling the number of parameters passed, avoiding passing unrelated data, keeping interfaces small and well-defined, and communicating through explicit interfaces rather than shared state.

PART B — [من قائمة مقابلة بمطلق التصاعدي] (Approx. 5 Questions)

(from the 6-real-exam sweet) — **Source:** مقابلة بمطلق التصاعدي — 6 سنوات

NEW questions written in the real-exam phrasing, derived from the 6 recurring axes in `ALGAE_MASTERING_Software_Engineering_Interview_Questions.docx` — Coupling & Cohesion · OOD Process · Data Design · Quality. Mark estimates in the header.

B1 (1988) — Define software architecture. Explain why it may be necessary to design the system architecture before the actual programming begins.
Model answer: Software architecture is the high level structure of a system, the organization of its components, their interrelationships, and the principles that guide their design and evolution. It is not a diagram but a set of decisions about decomposition, interfaces and responsibility. It must precede detailed specification because it is the **earliest point at which the major quality attributes** — **modular boundaries, interfaces and data flow** — structures cannot be chosen coherently until the overall structure is fixed. If detailed design begins first, its decisions will be made without a framework to guide them, and the result will be hard to integrate and change later. Hence the rule below — the architecture is also the framework within which the requirements themselves become implementable. Furthermore, requirements can only be fully validated once a candidate architecture exists to test them against feasibility, performance and maintainability. Changing the architecture late is by far the most expensive kind of change, because it invalidates everything beneath it. Hence the architecture should be designed and reviewed before detailed design begins.

B2 (1988) — Describe the characteristics and criteria for a good design, and explain how each is achieved.

B3 (1988) — Explain the functional concepts/principles of software design: abstraction, modularity, information hiding and encapsulation.
Model answer: Abstraction is the elimination of the irrelevant and the amplification of the essentials; it lets a designer express at one level of detail without being distracted by the lower-level implementation. It is named procedure whose body is hidden: what is done, not how. **Information hiding** is the separation of the external aspects of an entity from its internal implementation, so that the implementation can change without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients.

B4 (1988) — Explain the characteristics of a good module. Explain how each of these characteristics is achieved.
Model answer: Abstraction is the elimination of the irrelevant and the amplification of the essentials; it lets a designer express at one level of detail without being distracted by the lower-level implementation. It is named procedure whose body is hidden: what is done, not how. **Information hiding** is the separation of the external aspects of an entity from its internal implementation, so that the implementation can change without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients.

B5 (1988) — Explain the fundamental concepts/principles of software design: abstraction, modularity, information hiding and encapsulation.
Model answer: Abstraction is the elimination of the irrelevant and the amplification of the essentials; it lets a designer express at one level of detail without being distracted by the lower-level implementation. It is named procedure whose body is hidden: what is done, not how. **Information hiding** is the separation of the external aspects of an entity from its internal implementation, so that the implementation can change without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients.

B6 (1988) — Explain the characteristics of a good module. Explain how each of these characteristics is achieved.
Model answer: Abstraction is the elimination of the irrelevant and the amplification of the essentials; it lets a designer express at one level of detail without being distracted by the lower-level implementation. It is named procedure whose body is hidden: what is done, not how. **Information hiding** is the separation of the external aspects of an entity from its internal implementation, so that the implementation can change without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients. **Modularity** is the property of a system that can be decomposed into well-defined, manageable units with well-defined interfaces among them. **Encapsulation** is the grouping together of data and operations that act on it, and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients.

B7 (1988) — Explain the following terms: (a) Object, (b) Class, (c) Method, (d) Message.
(a) Object — an entity able to save a state (information) and offering a number of operations (behaviors) to examine or affect that state; an object is characterized by its operations and a state that remembers their effect. **(b) Class** — a template for objects, describing the operations and the identity of the target object, the name of the requested operation and any other information needed; messages are often implemented as procedure or function calls. **Abstraction** — the elimination of the irrelevant and the amplification of the essentials, used to decide on what is important. **Inheritance** — the mechanism by which one class (subclass/derived classes) inherit state and behavior from a high level class (superclass/base class), created by abstracting common features from a set of classes. **Encapsulation** is the grouping together of data and operation and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients. **(c) Method** — the implementation of an operation. **(d) Message** — a request to an object to execute one of its methods.

B8 (1988) — Can we have inheritance without polymorphism? Explain the design guideline for good polymorphic classes.
Model answer: Yes — inheritance is the mechanism by which a subclass acquires state and behaviour from a superclass, with no polymorphism it is still useful for code reuse and for modeling "is-a" relationships, but the inheritance hierarchy is then purely a tool for sharing implementation, not for substitution. A subclass can inherit attributes and operations without ever overriding them, so inheritance can exist as a mechanism that the rest of the system never relies on. **Polymorphism** is the ability to treat objects of different types through a common interface, with each one responding in its own way, and it is what makes inheritance truly powerful. Without polymorphism, inheritance is not the enabling mechanism through which polymorphic substitution is implemented, so the two are not the same thing. **Guidelines for polymorphic classes:** give each class a single coherent responsibility; encapsulate — hide attributes and expose a small, complete, well-named, stable interface; maximize cohesion inside and minimize coupling between classes; depend on abstractions (interfaces) rather than concrete classes; use inheritance only for genuine "is-a" generalization and avoid deep fragile hierarchies (the Shape/Square/Triangle example is the right case); and design for interface stable — because once reuses depend on it, changing it becomes very hard.

B9 (1989) — Write a short note on data design, and list the guidelines for data design.
Model answer: Data design is the process of translating the analysis model into the data structures required for implementation — it refines the data objects and the relationships among them into the structures the software will actually use internal data structures, files, tables, databases and data stores), and it

C2 (1990) — Compare function-oriented and object-oriented design for a project with rapidly changing requirements. Which is more suitable and why?
Model answer: Function-oriented design organizes the system by **what it does** — processes that transform data, implemented as a hierarchy of modules decomposed with DFDs and structure charts, and refined top-down as a tree of functions. **Object-oriented design** organizes the system by **what it is** — objects that encapsulate their own data and operations, communicating by sending messages, documented with UML — and the two views are explicitly **orthogonal**. **Function-oriented design** is the design of software where the units are functions — operations that transform data, organized into a hierarchy of modules and refined top-down. **Object-oriented design** is the design of software where the units are objects — encapsulated data items, instead the units that are to be manipulated by the program — it is orthogonal to function oriented design. Object oriented design is the design of software where the units are objects — encapsulated data items, instead the units that are to be manipulated by the program — it is orthogonal to function oriented design.

C3 (1990) — "Modularity is the single attribute that makes a program intellectually manageable." Discuss this claim with reference to the modularity/cost curve and the cohesion-coupling principle.
Model answer: This claim (Slide 13) is that modularity — the decomposition of software into well-defined, manageable units with well-defined interfaces — is the single attribute enabling intellectual management of programs. The **modularity/cost curve** (Slide 13) explains both halves of the word "manageable": adding units lowers the cost of developing each piece, but it raises the cost of integrating the pieces, so total effort as a function of the number of units is **U-shaped**, with a **Region of Minimum Cost** — there is a sweet spot in the number of modules, and both too few and too many are expensive. **Cohesion** is a measure of how well a module fits together. A module with **low cohesion** tends to group together unrelated ideas and operations that have little to do with one another. **High cohesion** tends to group together ideas that are related to one another and that work together to provide well-bounded behavior. Modules with high cohesion tend to be more reusable than modules with low cohesion.

C4 (1990) — A designer proposes to eliminate all global data and replace it with a single shared "context" object passed everywhere. Evaluate this proposal using the coupling taxonomy.
Model answer: This is a well-intentioned but poor trade. Global data is **common coupling** — each module's share a global data area, so changing the common data forces a trace of every module that accesses it (slide 21) — but solving it by passing one shared context object everywhere is **stamp coupling** — a composite data structure is passed between modules. Stamp coupling is better than common coupling because the dependency is explicit and passed between the modules to another, slide 20) the caller and callee are now bound to the whole structure, so any change to which fields the context holds can ripple to every function in the system, and the coupling is invisible in the interfaces — the context is used to carry flags or make assumptions. **control coupling** (module A passes control information to module B, typically a flag that tells B what to do) — the caller knows about the internal structure of the callee, and the callee is no longer a reusable unit. [5]

C5 (1990) — Design a small module structure (5–7 modules) for the University Library "issue book" use case, and justify each choice using coupling and cohesion.
Model answer: Using the library-clue study (slide 61) — issue of books with a limit of 4 Book-book and 2 General-books for 15 days, two-rule overdue fine, and reservation — a defensible module structure is: (1) **issue_book** — the top-level transaction module that sequences the use case (high level control); (2) **validate_student** — checks the student's eligibility and current holdings against the limit; (3) **validate_book** — checks the book's status (General-Book/book) and availability; (4) **compute_due_date** — takes the current system date and returns the return date, 15 days later; (5) **record_issue** — writes the transaction to the file; and (6) **print_slip** — produces the stamping device, and optionally (7) **log_transaction** — writes the auditing entry. **Justification by cohesion:** each module has **functional cohesion** — it does one well-defined thing, and its name describes exactly that thing. "Things done at issue time" would have produced only **temporal cohesion**, which is why the decomposition is by task and not by time. **Justification by coupling:** the module communicates by **data coupling** —

each book passes student-id and book-id to the validators, which return booleans/status values. **Justification by information hiding:** the caller of validate_book does not know or care how it checks availability — it could consult flat-file (sequential organization), and no module reads another's internals. (exist content coupling.) For validation, module A passes data to module B, and B returns a result. Data coupling is the safest form of coupling. [5]

C6 (1990) — Compare Structured Programming (Dijkstra) with Modular Programming (Parnas). Which is more relevant to a team building a large system?
Model answer: Structured Programming (Dijkstra) is about **how to write a single module** — it argues that any program can be written using only three control constructs: **sequence**, **selection** (if-then-else), and **iteration** (while/for), and that the use of **goto** should be abolished because it produces unstructured, hard-to-follow code. Its concern is the internal logic of one unit. **Modular Programming** (Parnas) is about **how to divide a system into units** — it argues that a system should be decomposed into modules, each hiding a design decision that is likely to change, so that a change affects only one module. Its concern is the external structure of many units. **Structured Programming** (Dijkstra) is about **how to write a single module** — it argues that any program can be written using only three control constructs: **sequence**, **selection** (if-then-else), and **iteration** (while/for), and that the use of **goto** should be abolished because it produces unstructured, hard-to-follow code. Its concern is the internal logic of one unit. **Modular Programming** (Parnas) is about **how to divide a system into units** — it argues that a system should be decomposed into modules, each hiding a design decision that is likely to change, so that a change affects only one module. Its concern is the external structure of many units.

C7 (1990) — Compare the design notations and documents used across the chapter (structure charts, DFDs, interaction-control diagrams, pseudocode, functional procedure logic, and the SDD's four views), and explain what a complete design package must contain.
Model answer: The chapter presents notations in two families, and the SDD binds them together. The **function-oriented** (slides 40–49) are **data flow diagrams**, which show the functional/process view (data entering, being transformed, and leaving) and are then refined into structure charts, which define the data items and structures consistently; **structure charts**, which partition the system into black box components, optimize the interfaces, and depict the hierarchy of control; and **pseudocode**, **functional procedural structure**, in which IMMER module is described in English, with structured constructs and data interaction modules — slide 44–45). **pseudocode/PDL**, short-structured English using IF/Then/Else, While/Do and End, usable in both preliminary and detailed design. **Functional procedure logic**, which specifies the algorithm and

C8 (1990) — Explain the following terms: (a) Object, (b) Class, (c) Method, (d) Message.
(a) Object — an entity able to save a state (information) and offering a number of operations (behaviors) to examine or affect that state; an object is characterized by its operations and a state that remembers their effect. **(b) Class** — a template for objects, describing the operations and the identity of the target object, the name of the requested operation and any other information needed; messages are often implemented as procedure or function calls. **Abstraction** — the elimination of the irrelevant and the amplification of the essentials, used to decide on what is important. **Inheritance** — the mechanism by which one class (subclass/derived classes) inherit state and behavior from a high level class (superclass/base class), created by abstracting common features from a set of classes. **Encapsulation** is the grouping together of data and operation and facing the implementation to subclasses, so it is possible to change the implementation without affecting clients. **(c) Method** — the implementation of an operation. **(d) Message** — a request to an object to execute one of its methods.

C9 (1990) — Can we have inheritance without polymorphism? Explain the design guideline for good polymorphic classes.
Model answer: Yes — inheritance is the mechanism by which a subclass acquires state and behaviour from a superclass, with no polymorphism it is still useful for code reuse and for modeling "is-a" relationships, but the inheritance hierarchy is then purely a tool for sharing implementation, not for substitution. A subclass can inherit attributes and operations without ever overriding them, so inheritance can exist as a mechanism that the rest of the system never relies on. **Polymorphism** is the ability to treat objects of different types through a common interface, with each one responding in its own way, and it is what makes inheritance truly powerful. Without polymorphism, inheritance is not the enabling mechanism through which polymorphic substitution is implemented, so the two are not the same thing. **Guidelines for polymorphic classes:** give each class a single coherent responsibility; encapsulate — hide attributes and expose a small, complete, well-named, stable interface; maximize cohesion inside and minimize coupling between classes; depend on abstractions (interfaces) rather than concrete classes; use inheritance only for genuine "is-a" generalization and avoid deep fragile hierarchies (the Shape/Square/Triangle example is the right case); and design for interface stable — because once reuses depend on it, changing it becomes very hard.

C10 (1990) — Write a short note on data design, and list the guidelines for data design.
Model answer: Data design is the process of translating the analysis model into the data structures required for implementation — it refines the data objects and the relationships among them into the structures the software will actually use internal data structures, files, tables, databases and data stores), and it

C11 (1991) — Explain a two-level process of design. Why should system design be finished before detailed design, rather than starting detailed design right after the requirements specification?
Model answer: Design has two levels — **architectural (high-level/system) design** partitions the system into major components, and defines their relationships, responsibilities and interfaces. **Detailed design** specifies the internal logic of each component — algorithms, data structures, and pseudocode. The two-level process is the **separation of concerns** — it separates the question of what